

Department of Computer Science and Engineering

Course Code &Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment 2: Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record system
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Maximum Marks	50
Duration	1 week
Mode	Individual
Student Reg. No.	192472170
Student Name	G. Sirisha

SECTION A – ALGORITHM DESIGN AND EFFICIENCY ANALYSIS

Scenario – University Student Record Retrieval: A university maintains a large academic database containing student records, each identified by a unique Registration Number. Faculty and administrative staff frequently need to determine whether a particular student record exists and retrieve the corresponding information. The database may contain 1,000 → 10,000 → 100,000 → 1,000,000+ records. Investigate, design and compare efficient algorithmic approaches for locating a given registration number as the database grows. Your solution must remain practical for both small and very large datasets, and any preprocessing performed on the records must be included in your efficiency analysis.

Q1. Design the Algorithm (5 Marks)

Design and propose at least three meaningfully different algorithmic strategies for locating a given registration number. Include at least one iterative and at least one recursive approach; iterative and recursive implementations of the same underlying algorithm will not be counted as two different approaches. For each approach, explain the basic idea, give pseudocode or an appropriate flowchart, identify the design strategy and any preprocessing required (such as sorting or restructuring the data), and briefly justify why you selected it for investigation. The appropriateness and diversity of your chosen strategies form part of the assessment.

For finding a student using a unique Registration Number, three different approaches can be used.

Approach 1: Linear Search

Idea: Check the registration numbers one by one until the required number is found.

Design strategy: Brute-force / sequential search.

Preprocessing: None.

Pseudocode:

LinearSearch(A, key)

for i = 0 to n-1

if A[i].regNo == key

return A[i]

return NOT_FOUND

This approach is simple and useful when the database is small or unsorted.

Approach 2: Recursive Binary Search

Idea: Sort the records first. Compare the required number with the middle record and continue searching only in the appropriate half.

Design strategy: Divide and conquer.

Preprocessing:

$O(n \log n)$

for sorting.

Pseudocode:

BinarySearch(A, low, high, key)

if low > high

return NOT_FOUND

mid = (low + high) / 2

if A[mid].regNo == key

return A[mid]

if $key < A[mid].regNo$

return $BinarySearch(A, low, mid-1, key)$

else

return $BinarySearch(A, mid+1, high, key)$

It is suitable for large datasets because half of the remaining records are eliminated after each comparison.

Approach 3: Hash-Based Search

Idea: Create a hash table using Registration Number as the key.

Design strategy: Hashing / indexing.

Preprocessing:

$O(n)$

expected time to build the hash table.

Pseudocode:

BuildHashTable(A)

create empty hash table

for each record in A

$table[record.regNo] = record$

return table

Search:

HashSearch($table, key$)

if key exists in table

return table[key]

else

return NOT_FOUND

This is useful when the system receives a large number of exact-match searches.

Q2. Identify the Basic Operation (2 Marks)

For each proposed algorithm, identify the basic operation that predominantly determines its execution time. State what the operation is, where it occurs in the algorithm, why it is appropriate for efficiency analysis, and how the number of times it executes changes as the input size n increases. Do not simply name an operation; relate it to the behaviour of the algorithm.

Linear Search

The main operation is the comparison:

$$A[i].regNo == key$$

It may execute once in the best case and up to $\backslash(n\backslash)$ times in the worst case.

Binary Search

The main operation is the comparison between the key and the middle registration number.

Each comparison removes approximately half of the remaining records.

Hash Search

The important operation is the hash-table lookup. With a good hash function, the expected number of accesses is approximately constant.

Q3. Derive the Time-Complexity Function Mathematically (6 Marks)

Derive the time-complexity function for each approach using mathematical analysis. Show the input size n , the number of times the basic operation executes, how the operation count changes with n , the mathematical expression for the running time, any preprocessing cost, and the overall cost when preprocessing and searching are considered together. Merely stating that an algorithm is $O(n)$, $O(\log n)$, etc. will not receive full credit.

1. Linear Search

For n records:

Best case

The required record is first.

$$C_{best} = 1$$

Therefore:

$$\boxed{\Theta(1)}$$

Worst case

The record is last or absent.

$$C_{worst} = n$$

Therefore:

$$\boxed{\Theta(n)}$$

Average case

Assuming the record can occur at any position:

$$C_{avg} = 1 + 2 + \dots + \frac{n}{n}$$

$$= \frac{n(n+1)}{2n}$$

$$= \frac{n+1}{2}$$

Therefore:

$$\boxed{\Theta(n)}$$

2. Binary Search

Sorting is performed first:

$$P(n) = \Theta(n \log n)$$

During searching, the problem size becomes:

$$n, \frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \dots$$

After k comparisons:

$$\frac{n}{2^k} = 1$$

Therefore:

$$2^k = n$$

$$k = \log_2 n$$

Hence:

$$S(n) = \Theta(\log n)$$

If sorting and one search are both counted:

$$T(n) = \Theta(n \log n) + \Theta(\log n)$$

Therefore:

$$\boxed{T(n) = \Theta(n \log n)}$$

If the sorted data is reused for many searches, each subsequent search costs only:

$$\boxed{\Theta(\log n)}$$

3. Hash Search

Building the table requires processing every record:

$$P(n) = \Theta(n)$$

After that, a lookup takes expected constant time:

$$S(n) = \Theta(1)$$

Therefore:

$$T(n) = \Theta(n) + \Theta(1)$$

$$\boxed{T(n) = \Theta(n)}$$

including index creation.

For subsequent searches:

$$\boxed{\Theta(1)}$$

expected time.

Q4. Analyse Best, Average and Worst-Case Behaviour (4 Marks) For each algorithm, explain which input condition produces the best case, the average case and the worst case, how the

number of operations changes in each case, and why these differences occur. Support your analysis with mathematical reasoning; a table of complexity values alone is not sufficient.

Algorithm	Best	Average	Worst
Linear Search	1	$(n+1)/2$	n
Binary Search	1	$\Theta(\log n)$	$\Theta(\log n)$
Hash Search	O(1) expected	O(1) expected	O(n)

Linear Search

Best case occurs when the first record matches. Worst case occurs when the record is last or absent.

Binary Search

Best case occurs when the middle record is the required student. In average and worst cases, the search repeatedly halves the dataset, giving logarithmic behaviour.

Hash Search

With a good hash function, the required record is normally reached directly. Heavy collisions can increase the number of comparisons, giving $\mathcal{O}(n)$ in the theoretical worst case.

Q5. Perform Asymptotic Analysis (3 Marks) Express and compare the growth behaviour of your algorithms using O , Ω and/or Θ wherever relevant. Explain how each algorithm behaves as n becomes very large, how their growth rates differ, and what this growth means when the dataset increases from 1,000 to 1,000,000+ records. Interpret the asymptotic behaviour in the context of the application rather than merely stating the notation.

The three methods have different growth rates:

$$\text{Linear Search} = \Theta(n)$$

$$\text{Binary Search} = \Theta(\log n)$$

$$\text{Hash Search} = \Theta(1)$$

expected after preprocessing.

For example:

$$\log_2(1000) \approx 10$$

while:

$$\log_2(1,000,000) \approx 20$$

Binary search therefore increases very slowly even when the database becomes much larger.

Linear search, however, can increase from around 1,000 comparisons to 1,000,000 comparisons.

Hash search remains approximately constant per request.

Q6. Formulate the Recurrence Relation (3 Marks)

For the recursive approach, formulate a recurrence relation directly from the behaviour of your algorithm. Identify the input size of the original problem, the size of the recursive subproblem, the number of recursive calls, the work performed outside the recursive call and the base case, and show how these components lead to the recurrence. Do not simply write a known recurrence formula.

For recursive binary search:

- Original problem size = n
- One recursive subproblem = $n/2$
- Number of recursive calls = 1
- Work outside recursion = one comparison, which is constant
- Base case = one remaining record

Therefore:

$$T(n) = T(n/2) + c$$

with:

$$T(1) = c$$

Q7. Solve the Recurrence Relation (4 Marks)

Solve the recurrence obtained in Q6 using the substitution method, iteration method, Master Theorem or another appropriate analytical technique. Explain why the selected method is applicable, show the steps involved, state the resulting complexity and interpret what it indicates about the behaviour of the recursive algorithm as n increases.

Given:

$$T(n) = T(n/2) + c$$

Substitute repeatedly:

$$T(n) = T(n/2) + c$$

$$= T(n/4) + 2c$$

$$= T(n/8) + 3c$$

After k steps:

$$T(n) = T(n/2^k) + kc$$

The base case occurs when:

$$n/2^k=1$$

Therefore:

$$2^k = n$$

$$k = \log_2 n$$

Hence:

$$T(n) = T(1) + c \log_2 n$$

Therefore:

$$\boxed{T(n) = \Theta(\log n)}$$

This shows that recursive binary search remains efficient even for very large databases.

Q8. Implement and Test the Algorithms (4 Marks)

Implement all the proposed approaches and test them on datasets of approximately 1,000, 10,000, 100,000 and 1,000,000 records. Ensure that all algorithms produce the correct result, the same or equivalent datasets are used, testing conditions are kept reasonably consistent, appropriate test cases are included, and the implementations correspond to the algorithms analysed. The purpose is to generate evidence that validates your theoretical analysis, not merely to demonstrate working code.

Java Implementation

```
import java.util.*;
```

```
public class StudentSearch {  
  
    static class Student {  
  
        int regNo;  
  
        Student(int regNo) {  
  
            this.regNo = regNo;  
  
        }  
  
    }  
  
    // Linear Search  
  
    static Student linearSearch(Student[] a, int key) {  
  
        for (Student s : a) {  
  
            if (s.regNo == key)  
  
                return s;  
  
        }  
  
        return null;  
  
    }  
  
    // Recursive Binary Search  
  
    static Student binarySearch(Student[] a, int low,  
  
                                int high, int key) {  
  
        if (low > high)  
  
            return null;  
  
        int mid = low + (high - low) / 2;  
  
        if (a[mid].regNo == key)  
  
            return a[mid];  
  
        if (key < a[mid].regNo)  
  
            return binarySearch(a, low, mid - 1, key);  
  
    }  
  
}
```

```

        return binarySearch(a, mid + 1, high, key);
    }

    // Create dataset

    static Student[] createData(int n) {

        Student[] a = new Student[n];

        for (int i = 0; i < n; i++)

            a[i] = new Student(100000 + i);

        return a;
    }

    public static void main(String[] args) {

        int[] sizes = {1000, 10000, 100000, 1000000};

        for (int n : sizes) {

            Student[] data = createData(n);

            int key = data[n - 1].regNo;

            long start = System.nanoTime();

            Student r1 = linearSearch(data, key);

            long linearTime = System.nanoTime() - start;

            start = System.nanoTime();

            Student r2 = binarySearch(data, 0, n - 1, key);

            long binaryTime = System.nanoTime() - start;

            HashMap<Integer, Student> map = new HashMap<>();

            start = System.nanoTime();

            for (Student s : data)

                map.put(s.regNo, s);

            long hashBuildTime = System.nanoTime() - start;

```

```

start = System.nanoTime();

Student r3 = map.get(key);

long hashSearchTime = System.nanoTime() - start;

System.out.println("\nN = " + n);

System.out.println("Linear: " + linearTime + " ns");

System.out.println("Binary: " + binaryTime + " ns");

System.out.println("Hash Build: " + hashBuildTime + " ns");

System.out.println("Hash Search: " + hashSearchTime + " ns");

    }

}

}

```

Test sizes:

1000, 10000, 100000, 1000000

The same dataset and search key should be used as far as possible to make the comparison fair.

Q9. Record Experimental Performance (3 Marks) Collect experimental data for each algorithm across the different input sizes, recording measures such as execution time, number of comparisons, number of key operations and other justified measures. Present the results using tables, graphs or performance plots so that the approaches can be compared easily.

The theoretical comparison count is:

Records	Linear worst case	Binary worst case	Hash expected
1,000	1,000	~10	~1
10,000	10,000	~14	~1
100,000	100,000	~17	~1
1,000,000	1,000,000	~20	~1

The actual execution-time values should be recorded by running the Java program.

Experimental table

n	Linear worst comparisons	Binary worst comparisons	Hash expected lookup
1,000	1,000	10	~1
10,000	10,000	14	~1
100,000	100,000	17	~1
1,000,000	1,000,000	20	~1

Q10. Compare Theory with Experimental Behaviour (4 Marks)

Compare your theoretical analysis with your experimental results. Discuss whether the observed growth agrees with the theoretical complexity, which algorithm scales better as n increases, any differences between theoretical prediction and actual execution, and possible reasons for those differences. Interpret the results rather than reproducing tables or graphs.

The expected experimental behaviour should follow the theoretical analysis.

Linear search should show a noticeable increase in execution time because its work grows directly with $\sqrt[n]{n}$.

Binary search should remain almost stable because the number of comparisons increases only logarithmically.

Hash search should also remain nearly constant after the hash table is constructed.

Small differences between theory and measured execution time can occur because of CPU speed, memory access, JVM optimization, garbage collection and other running applications.

Therefore, the experiment is expected to confirm the general growth patterns rather than produce exactly the same numerical values predicted by the mathematical model.

Q11. Recommend and Defend the Final Design (4 Marks)

Recommend the most appropriate algorithmic solution for the university student-record system. Explain why it is more appropriate than the alternatives, its theoretical efficiency, its experimentally observed performance, any preprocessing cost, relevant trade-offs or limitations, and whether it remains practical as the database grows. Base the

recommendation on your own analysis and experiments; identifying the algorithm with the lowest Big-O complexity is not sufficient.

For the university system, I recommend Hash-Based Search as the primary solution.

The reasons are:

1. Registration Number is unique and suitable as a hash key.

2. Index construction requires expected:

$$\Theta(n)$$

3. Each search requires expected:

$$\Theta(1)$$

4. It is highly suitable when many students are searched repeatedly.

5. The database can be indexed once and reused.

Comparison

Method	Preprocessing	Search	Suitability
Linear	None	$O(n)$	Small datasets
Binary	$O(n \log n)$	$O(\log n)$	Large sorted data
Hash	$O(n)$ expected	$O(1)$ expected	Frequent searches

Hashing uses additional memory and can suffer from collisions, but for exact Registration Number searches its performance is very attractive.

Therefore:

Recommended: Hash-Based Search

SECTION B – DAY-4 DESIGN CHALLENGE

Q12. Re-evaluate and Adapt the Design (8 Marks)

Changed requirement: the university now needs to process 50,000 search requests per hour on a database containing 1,000,000 student records, but new records are added only once per day. Re-evaluate your original design under these operating conditions and decide whether to RETAIN, MODIFY or REDESIGN it.

New conditions:

$$n = 1,000,000$$

and:

$$50,000 \text{ searches/hour}$$

Records are updated only once per day.

(a) Impact Analysis (2 Marks): Discuss what has changed from the original problem, which original assumptions are affected, why the frequency of searches is now important, and why the relatively low frequency of record updates may influence the design.

The important change is that the number of searches is now very high.

There are:

$$50,000$$

searches every hour.

At the same time, the database changes only once a day. Therefore, the university can spend some time creating an index and reuse it for many searches.

The system is now read-heavy and update-light.

(b) Evaluation of the Original Design (2 Marks): Evaluate its expected behaviour under 50,000 searches per hour, whether its preprocessing cost is acceptable under the new conditions, and whether another approach could give better overall performance.

Linear Search

Worst-case work for one search:

$$1,000,000$$

For 50,000 searches:

$$50,000 \times 1,000,000$$

$$= 50,000,000,000$$

comparisons per hour.

This is not suitable.

Binary Search

Approximately:

$$\log_2(1,000,000) \approx 20$$

comparisons per search.

Therefore:

$$50,000 \times 20$$

$$= 1,000,000$$

comparisons per hour.

This is practical.

Hash Search

Expected:

$$O(1)$$

per search.

Thus, 50,000 searches require approximately:

$$\boxed{O(50,000)}$$

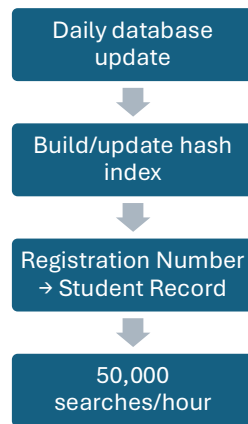
expected lookup operations.

(c) Adaptation or Redesign (2 Marks): If you modify or redesign the solution, describe the revised strategy, explain what has changed and determine the effect on computational complexity. If you retain it, provide sufficient technical justification for why no redesign is necessary.

Decision: MODIFY

I would modify the design to maintain a daily-built hash index.

The process becomes:



Daily index construction:

$$\Theta(n)$$

expected.

Each search:

$$\Theta(1)$$

expected.

For q searches:

$$T(n, q) = \Theta(n + q)$$

expected.

(d) Evidence-Based Defence (2 Marks): Defend your final decision using mathematical complexity analysis, experimental results where appropriate, search frequency, preprocessing cost, dataset size and relevant trade-offs. The decision must follow logically from your analysis.

For one hour:

Linear Search

$$50,000 \times 1,000,000 = 50 \text{ billion}$$

✗ Not practical.

Binary Search

$$50,000 \times 20 = 1,000,000$$

comparisons.

✓ Practical.

Hash Search

Expected:

$$50,000 \times O(1)$$

✓ Most efficient for repeated exact searches.

Since new records are added only once per day, the cost of building the hash index is easily justified by the large number of searches performed during the day.

Final Decision

MODIFY – Use a daily hash – based Registration Number index

This design gives:

$$\Theta(n)$$

expected daily preprocessing and:

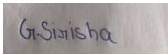
$$\Theta(1)$$

expected time for each search.

Therefore, it is the most practical choice for 1,000,000+ student records and 50,000 searches per hour.

Code of Conduct

I **G. Sirisha (192472170)** (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

Date: 07-10-26